Lazy: API decisions and tradeoffs

Decision record for Java architects — updated 5 October 2026 — branch `prototype/lazy-one-class`.

This document records the design developed in this discussion and embodied in the specification of `java.lang.util.Lazy` in this branch. It summarizes decisions and consequences; the source Javadoc remains the normative contract for this prototype. “Adopted” means adopted in this branch, not approved for the Java platform or committed to a release.

The starting point was the work associated with JDK-8336664, including `LazyConstant` and the `List.ofLazy`, `Map.ofLazy`, and `Set.ofLazy` factories, followed by Brian Goetz's email reproduced in the discussion. The email supplied the overall direction. Defects in the subsequent naming, typing, failure, memory, and collection decisions described here should be blamed on Per only. The pros and cons below are architectural assessments, not quotations or evidence of wider consensus.

The architectural direction

The adopted design is one non-generic, final utility class, `java.lang.util.Lazy`, with two factory families. Cache factories own storage and return familiar `Supplier`, `Function`, `List`, `Map`, or `Set` interfaces. Updater factories use a caller-owned field, array element, or memory-segment location and return a functional-interface implementation whose method locates that storage. Both families defer computation, but their storage obligations and available policies differ. Factory names include `Cache` or `Updater` so callers can see which contract applies before reading an overload. The advanced updater path starts with `ofUpdater(Class<C>)` and configures its storage and policy through an immutable builder.

Two principles from Brian's proposal drive the design. First, laziness has several legitimate meanings: initialization with irreversible side effects needs different coordination from a repeatable calculation. Second, ordinary users should not need to understand VM stability machinery to use the API correctly. Constant folding remains an optimization opportunity underneath a usable library abstraction.

One class gives laziness a single discovery point and lets the specification explain the common principle before separating the two storage models. The cost is a larger class whose users must still distinguish a cache's owned storage from an updater's reserved-default-value protocol. The new `java.lang.util` package is neutral with respect to the former `java.util` and `java.lang.invoke` homes, but introduces another exported `java.base` package and requires migration from both old class names. The design covers selected useful combinations rather than every combination of coordinates, storage, failure policy, and memory ordering.

The cache family offers two modes, a default `COMPUTE_ONCE` scalar-supplier factory, and acquire/release publication. The updater family offers three modes, including `PLAIN`; the advanced updater builder defaults to `PUBLISH_ONCE` and a public lookup. Explicit `withLookup` supplies authority for non-public fields, while narrower static- and instance-field convenience factories derive authority from their caller. A direct-array convenience factory needs no lookup. The distinct `CacheMode` and `UpdaterMode` enums prevent passing an updater-only policy to a cache factory. The main costs are builder steps for advanced shapes, possible duplicate work or permanent failure depending on the family, and extra obligations for caller-owned storage.

How the design evolved

Stage	Step taken	Step not taken
API organization	Followed Brian's one-class direction through `Lazy`, then temporarily separated `java.util.LazyCaches` and `java.lang.invoke.LazyUpdaters`; this branch reunites both families in `java.lang.util.Lazy`.	Did not expose the earlier family of `OfAbsolute`, `OfRelative`, `OfIndexed`, and `OfIndexedRelative` interfaces, or keep two public facades as aliases.
Factory usability	Replaced `BoundFunction` with `Function`, used `ofScalar` for both scalar cache shapes, then named cache factories `ofScalarCache`, `ofListCache`, `ofMapCache`, and `ofSetCache`. Earlier advanced `ofLocationUpdater` / `ofArrayUpdater` overloads were consolidated into `ofUpdater(Class<C>)` with an immutable builder. Retained `ofStaticFieldUpdater`, `ofFieldUpdater`, and a reference-array `ofArrayUpdater` convenience.	Did not retain bare `of`, `ofContext`, or multiple advanced factories split by storage kind and computation timing. The two `ofScalarCache` overload shapes still admit some method-reference ambiguity.

Stage	Step taken	Step not taken
Policy naming	Evolved the proposal's `InitMode` and the branch's `ONCE` / `RACE` terminology into `Lazy.CacheMode.COMPUTE_ONCE` / `PUBLISH_ONCE`; renamed updater `CAS` / `SYNCHRONIZED` to `PUBLISH_ONCE` / `SERIALIZED` to describe their guarantees.	Did not make the public mode names depend on a particular atomic instruction or locking mechanism, or offer independent retry, exclusion, and fencing switches.
Mode typing	Renamed the former independent `LazyCaches.Mode` and `LazyUpdaters.Mode` enums to `Lazy.CacheMode` and `Lazy.UpdaterMode`.	Did not merge the enums merely because both contain `PUBLISH_ONCE`, or expose updater-only modes to cache factories.
Updater mode selection	Initially required `UpdaterMode` as the first argument of the general factories. The builder now defaults to `PUBLISH_ONCE`, with `withMode` for explicit alternatives; the three narrow conveniences also fix `PUBLISH_ONCE`.	Did not add parallel no-mode overloads or a varargs mode parameter to every advanced factory. The default is visible in one builder contract.
Memory contract	Replaced the earlier volatile-access guarantee with acquire/release publication, including recorded failures.	Did not promise a total ordering across different lazy values or extra synchronization from repeated access.
Field access	Explored a named-field `Lazy.ofField`, then `LazyUpdaters.ofLocation`, then `Lazy.ofLocationUpdater`. The builder now selects an instance or static field with `withLocation(ownerClass, fieldName)`; the class token fixes loader identity and can also anchor inherited-field lookup. `Lazy.ofStaticFieldUpdater` and `Lazy.ofFieldUpdater` retain caller-sensitive access.	Did not bypass access checks or keep the later bytecode-cracked accessor-lambda scheme. A convenience cannot access a private field outside its caller's authority.
Array access	Extended updaters from one- and two-index access to any positive index count supported by the array rank and method arity. An array-valued selected field can be indexed, while an omitted `withLocation` selects an element of a directly supplied array. Retained the reference-array, one-index convenience that reuses `ReferenceArrayAccessor.Computer<T>`.	Did not retain the briefly introduced, duplicate `ObjIntFunction` interface, add a cache factory that returns an indexed-array accessor, infer arbitrary array expressions from lambdas, or promise index permutation.
Segment access	Added `withLocation(MemoryLayout, PathElement...)` for a value selected in a structured memory segment.	Did not restore the earlier `Lazy.ofSegment` overloads, support dereference path elements or `SERIALIZED` segment coordination, or silently weaken `PUBLISH_ONCE` for a layout without required atomic access.
Computation timing	Retained declaration-site factories that store one computation in the updater, and added use-site overloads that take a computation as the updater's first argument on each access.	Did not require a callback stored by the updater when different accesses need different computations. The caller must keep computations interchangeable for a shared location.
Contract completion	Added the set cache (originally `ofSet`) and specified collection views, entries, traversal failures, streams, callback retention, composition, and execution context.	Did not treat ordinary interface defaults as sufficient to define these interactions.
Implementation	After the API reviews, brought implementations into line and added focused and general collection regressions.	Did not remove the earlier lazy APIs, settle their migration policy, or establish performance claims from benchmarks in this work.

Brian's final sketch used `BoundSupplier` and `InitMode`, still overloaded `of`, and did not include a set cache. The current design follows that direction with substantive later refinements.

Current API surface

The following is a signature synopsis of `java.lang.util.Lazy`; bodies, annotations, and Javadoc are omitted. The `Cache` and `Updater` suffixes identify whether the factory provides storage or uses caller-owned storage. The two mode types make that distinction checkable by the compiler.

```
public enum CacheMode { COMPUTE_ONCE, PUBLISH_ONCE }
public enum UpdaterMode { PUBLISH_ONCE, SERIALIZED, PLAIN }
public static final class InitializationError extends Error { ... }
public interface ReferenceArrayAccessor<E> {
    interface Computer<E> {
        E apply(E[] array, int index);
    }
    E getOrCompute(Computer<E> computingFunction, E[] array, int index);
}

public static <T> Supplier<T> ofScalarCache(
        Supplier<? extends T> computingFunction);
public static <T> Supplier<T> ofScalarCache(
        CacheMode mode, Supplier<? extends T> computingFunction);

public static <C, T> Function<C, T> ofScalarCache(
        CacheMode mode, Function<? super C, ? extends T> computingFunction);

public static <T> List<T> ofListCache(
        CacheMode mode, int size, IntFunction<? extends T> computingFunction);
public static <K, V> Map<K, V> ofMapCache(
        CacheMode mode, Set<? extends K> keys,
        Function<? super K, ? extends V> computingFunction);
public static <E> Set<E> ofSetCache(
        CacheMode mode, Set<? extends E> elementCandidates,
        Predicate<? super E> computingFunction);

public static <T> Supplier<T> ofStaticFieldUpdater(
        Class<?> owner, String name, Supplier<T> computing);
public static <R, T> Function<R, T> ofFieldUpdater(
        Class<R> owner, String name, Function<R, T> computing);
public static <T> ReferenceArrayAccessor.Computer<T> ofArrayUpdater(
        Class<T> elementType, ReferenceArrayAccessor.Computer<T> computing);
public static <C> UpdaterBuilder<C> ofUpdater(Class<C> computationShape);

public interface UpdaterBuilder<C> {
    public UpdaterBuilder<C> withMode(UpdaterMode mode);
    public UpdaterBuilder<C> withLookup(MethodHandles.Lookup lookup);
    public UpdaterBuilder<C> withLocation(Class<?> ownerClass, String fieldName);
    public UpdaterBuilder<C> withLocation(
            MemoryLayout layout, MemoryLayout.PathElement... pathElements);
    public C build(C computingFunction);
    public <U> U buildUseSite(Class<U> useSiteShape);
}
```

`InitializationError` has a public `InitializationError(String message)` constructor and pertains only to `CacheMode.COMPUTE_ONCE`. Nine ready-made use-site accessor interfaces are nested in `Lazy` and described below. The nested `ReferenceArrayAccessor.Computer<T>` is also the return and computation type of the reference-array updater convenience; its `apply` method takes a `T[]` and one index. The nested `StaticObjectFieldSupplier extends Supplier<Object>` provides the concrete functional-interface token used by `ofStaticFieldUpdater`.

Factory	Unit of retained state	Access	Policy selection
`ofScalarCache(Supplier)` / `ofScalarCache(CacheMode, Supplier)`	One value per returned supplier	`get()`	Explicit mode, or `COMPUTE_ONCE` by default
`ofScalarCache(CacheMode, Function)`	One value per returned function, across all contexts	`apply(context)`	Explicit mode
`ofListCache`	One value per index	`get(index)` and ordinary list operations	One explicit mode, applied independently per element
`ofMapCache`	One value per copied key	`get(key)` and ordinary map operations	One explicit mode, applied independently per mapped value
`ofSetCache`	One Boolean membership result per copied candidate	`contains(candidate)` and ordinary set operations	One explicit mode, applied independently per candidate
`ofStaticFieldUpdater`	One caller-owned, caller-accessible static mutable field	Returned `Supplier<T>.get()`	`PUBLISH_ONCE` fixed by the convenience factory
`ofFieldUpdater`	One caller-owned, caller-accessible instance field per receiver	Returned `Function<R,T>.apply(receiver)`	`PUBLISH_ONCE` fixed by the convenience factory

Factory	Unit of retained state	Access	Policy selection
`ofUpdater(...).withLocation(ownerClass, fieldName).build(...)` / `buildUseSite(...)`	Caller-owned named field or array element reached through a named field	Returned interface's functional method	`PUBLISH_ONCE` by default; `withMode` selects another updater mode
`ofUpdater(...).build(...)` / `buildUseSite(...)`	Caller-owned direct-array element	Returned interface's functional method	`PUBLISH_ONCE` by default; `withMode` selects another updater mode
`ofUpdater(...).withLocation(layout, path).build(...)` / `buildUseSite(...)`	Value in a supplied memory segment	Returned interface's functional method	`PUBLISH_ONCE` by default if atomic access is supported; explicit `withMode(PLAIN)` may support more layouts; no `SERIALIZED`
`ofArrayUpdater(Class<T>, ReferenceArrayAccessor.Computer<T>)`	One element per index of each supplied reference array	Returned `ReferenceArrayAccessor.Computer<T>.apply(array, index)`	`PUBLISH_ONCE` fixed by the convenience factory

Among cache factories, only `ofScalarCache(Supplier)` supplies a default mode. There is no field, segment, or direct-array **cache** factory and no indexed-context cache shape. A list cache provides indexed access together with size, views, iteration, and streams; the price is a collection contract rather than a minimal coordinate interface. This is a surface-area choice, not a demonstrated claim of identical performance to an array. The advanced updater builder selects field-backed arrays through `withLocation(ownerClass, fieldName)` and direct arrays when no location is specified.

Consolidation and migration

This branch replaces the two preview facades rather than preserving forwarding aliases. The internal implementations remain separated by storage mechanism; the public class, package, naming, and specification become one. Clients migrate as follows:

Previous API	API in this branch
`java.util.LazyCaches`	`java.lang.util.Lazy`
`LazyCaches.Mode`	`Lazy.CacheMode`
`LazyCaches.InitializationError`	`Lazy.InitializationError`
`LazyCaches.ofScalar` / `ofList` / `ofMap` / `ofSet`	`Lazy.ofScalarCache` / `ofListCache` / `ofMapCache` / `ofSetCache`
`java.lang.invoke.LazyUpdaters`	`java.lang.util.Lazy`
`LazyUpdaters.Mode`	`Lazy.UpdaterMode`
`LazyUpdaters.ofLocation` / `ofArray`	Formerly `Lazy.ofLocationUpdater` / general `ofArrayUpdater`; now `Lazy.ofUpdater(Class<C>)` followed by builder configuration and `build` or `buildUseSite`
`LazyUpdaters` ready-made accessor interfaces	The same nested interface names under `Lazy`

The names make the storage owner visible in calls such as `Lazy.ofListCache(...)` and `Lazy.ofUpdater(...)`. This is useful because a returned updater and a returned cache can both have a `get`-like method while imposing different duties on the caller. A single class makes the families easier to discover and compare, but its specification is longer and its mode types remain separate. `java.lang.util` gives the merged class one neutral home; the cost is a new exported package and source and binary incompatibility with both previous facades. Clients must import `java.lang.util.Lazy` explicitly; automatic `java.lang` imports do not include subpackages. No claim of compatibility with code compiled against those names is made for this prototype branch. The pre-existing `LazyConstant` and `List.ofLazy`, `Map.ofLazy`, and `Set.ofLazy` APIs are outside this facade move. The new `@since 29` tags are provisional while this prototype builds on a feature-version-28 tree; release metadata must be aligned with the chosen integration release.

Standard interfaces, overload resolution, and inference

Returning `Supplier` and `Function` avoids introducing new access interfaces. Replacing `BoundFunction<C,T>.get(C)` with `Function<C,T>.apply(C)` improves interoperability and gives users ordinary composition. The corresponding cost is semantic: `Function` alone does not tell the reader where storage lives or whether distinct arguments receive distinct results.

The `ofScalarCache(CacheMode, Function)` overload owns **one slot**, not a cache indexed by context. For example:

```
Function<String, Integer> length =
        Lazy.ofScalarCache(Lazy.CacheMode.PUBLISH_ONCE, String::length);

length.apply("abc");    // Installs 3.
length.apply("hello");  // Returns the retained 3.
```

An instance-derived property normally needs its own `ofScalarCache(CacheMode, Function)` holder per instance. A static holder shared across instances would share one value. Supplying the context at access time permits a noncapturing callback; it does not eliminate the holder or guarantee allocation-free lambda execution. A null context is rejected even after initialization.

The shared `ofScalarCache` name makes the scalar category visible but again overloads supplier-shaped and context-shaped computations. Explicit zero-argument and one-argument lambdas generally distinguish them; overloaded or inexact method references—such as references to constructors with several arities—can make overload selection ambiguous. A destination result type does not generally rescue that selection. The earlier `ofContext` name avoided this ambiguity and made the access shape explicit, at the cost of another method name.

The remaining `var` issue is ordinary inference: Java cannot infer `String` merely from `s -> s.length()` when no context type is otherwise supplied. A typed destination, an explicitly typed lambda, a suitable method reference, or explicit type arguments supplies that information. For example:

```
var length = Lazy.ofScalarCache(Lazy.CacheMode.PUBLISH_ONCE,
                                (String s) -> s.length());
```

An additional `Class<C>` argument for the context-shaped overload was explored as an inference aid but was not adopted. It would make `var` examples easier to type, while adding an argument and another factory form despite there being no field type to validate in this case. This is a considered alternative, not a permanent rejection of any future convenience overload.

Composition retains the ordinary `Function` ordering: `compose` runs the preceding function on every application, and `andThen` runs the following function after every successful lazy access. Their results are not automatically cached. Composition itself records no additional initialization failure; nested lazy accesses still follow their own failure rules. This avoids silently changing familiar composition semantics, but requires callers to place expensive work inside the intended lazy boundary.

Two coordinated policies, named for their guarantees

Dimension	`COMPUTE_ONCE`	`PUBLISH_ONCE`
Intended use	Initialization, including side effects that must not repeat	Memoization where repeated computation is acceptable
Invocation count per slot	At most once; zero if never needed	Competing invocations and retries after failure are permitted
Competing access	Waits for the one invocation	May compute another candidate
Successful access	Returns the retained result	Returns the installed winner, including when its own candidate loses
Failure	Permanent	Propagated to that caller; not retained
Principal benefit	Prevents duplicate initialization and repeated side effects	Avoids requiring all callers to wait for a designated initializing computation
Principal cost	Waiting, permanent failure, and possible dependency deadlock	Duplicate work, discarded results, and side effects that may repeat

`COMPUTE_ONCE` states what is limited; `PUBLISH_ONCE` states what is committed. They are more informative than `ONCE` / `RACE` and avoid encoding a particular instruction such as CAS. “Once” does not require eager execution. The installed result is consistent across successful accesses to one slot, but the identity or contents of the winning candidate need not be predictable across executions.

Initialization versus memoization is an intended-use distinction, not a static-versus-instance restriction. Either mode can serve either location where the chosen factory supports it.

The two cache policies bundle exclusion and failure handling. The benefit is a small choice set; the cost is that a cache cannot request, for example, serialized initialization with retry or concurrent computation with sticky failure. `PUBLISH_ONCE` is not a promise that the API or computing function is lock-free or wait-free. Discarded results are not closed or otherwise disposed of by `Lazy`.

Mode types and the caller-owned-storage updater

`Lazy.CacheMode` and `Lazy.UpdaterMode` are distinct nested enums. `CacheMode` offers `COMPUTE_ONCE` and `PUBLISH_ONCE`; `UpdaterMode` offers `PUBLISH_ONCE`, `SERIALIZED`, and `PLAIN` for caller-owned storage. The same spelling `PUBLISH_ONCE` describes the shared successful-publication outcome, but the two families have different storage and failure protocols. A value from either enum cannot be passed to a factory expecting the other. Both retain enum conveniences such as `values()` and `switch` expressions; separate static imports of the two `PUBLISH_ONCE` constants would be ambiguous, so examples qualify the mode. `ofUpdater` defaults to updater `PUBLISH_ONCE`; the narrow updater conveniences fix that mode, while `withMode` changes a builder's choice before a terminal operation.

The updater's `PUBLISH_ONCE` replaces the earlier name `CAS`: it names the one installed value rather than the operation used to install it. `SERIALIZED` replaces `SYNCHRONIZED`: it promises one computation at a time per storage location without requiring a particular locking implementation. Unlike `Lazy.CacheMode.COMPUTE_ONCE`, `SERIALIZED` allows another access to retry after failure.

The builder's `withLocation(ownerClass, fieldName)` identifies an instance or static field using a `Class` token and a simple field name. It can also select an array element through an array-valued field when one or more `int` index coordinates follow the receiver (or form all storage coordinates for a static field). With no location selector, the computation shape's first storage coordinate must be a direct array and its remaining coordinates one or more `int` indices. In a use-site interface, the computing-function parameter comes before these storage coordinates. The index count must not exceed the array rank and remains subject to ordinary Java and JVM method-parameter limits. The interface fixes that count: `int...` erases to one `int[]` parameter and cannot stand in for several index parameters. `withLocation(layout, pathElements)` instead selects a segment value with its own coordinate rules and mode restrictions, described below. The three conveniences return `Supplier`, `Function`, and `ReferenceArrayAccessor.Computer`, respectively. Updaters reserve the selected location's default value to mean uninitialized and reject a computation that produces that value.

`Lazy.UpdaterMode`	Access and coordination	Failure	Principal benefit and cost
`PLAIN`	Plain read and write, without CAS or exclusion; each computing caller returns its own result	Propagated directly; retry remains possible	Least coordination, at the cost of possible overwrites, stale reads, and no safe-publication guarantee; plain `long`/`double` access may tear
`PUBLISH_ONCE`	Acquire observation and atomic release installation of a non-default value; competing candidates race and successful callers return the installed winner (materialized from the location for an address layout)	Propagated directly; retry remains possible	No per-receiver coordination state, at the cost of duplicate work and discarded candidates
`SERIALIZED`	Acquire read, release write, and one computing thread at a time for the same field and receiver (or static field), or physical array element, across updater objects; unavailable for segment locations	Propagated directly without being retained; a later or waiting caller may retry	Avoids concurrent computations, at the cost of waiting and auxiliary coordination state

`SERIALIZED` shares coordination state across updater objects for a field and tracks instance receivers by weak identity. For arrays of any rank, its key is the identity of the final container array and the final index. The preceding indices only traverse outer dimensions. Two outer arrays or owner fields that refer to the same selected container array therefore coordinate access to the same element, as do direct-array and field-backed updaters that reach it. If a computation throws or returns the reserved default value, the location remains uninitialized; the next caller to acquire that state may compute again. The state must remain shared after failure so waiting and newly arriving callers cannot compute concurrently. After success, the location retains the result and the coordination state may be discarded. Cleared receiver or container-array entries are removed on a later uninitialized access, so their metadata may remain until then or until class unloading.

Updaters sharing a physical storage location must all use the same mode and should have interchangeable computing functions. For a use-site updater, this also applies to callbacks supplied on different calls to the same updater. The location must not be reset or written directly while they are in use; compatible same-mode updaters may share it. `PLAIN` is appropriate only when callers accept its weaker outcomes or provide their own synchronization. It is available only to updater factories,

where the caller explicitly controls storage, without weakening the guarantees of cache factories in the same class. Aliased segment views are why `SERIALIZED` is initially rejected for segment locations: locking by the Java segment object's identity would not serialize accesses to the same physical bytes.

Failure policy: preserve the Error category without retaining the original failure

The discussion explicitly chose to propagate an initial `Error` directly and keep its failure permanent in `COMPUTE_ONCE`. Subsequent observations remain in the `Error` hierarchy, using the dedicated `Lazy.InitializationError` rather than `NoSuchElementException`.

Computation outcome	Access executing the computation in `COMPUTE_ONCE`	Access observing its recorded failure in `COMPUTE_ONCE`	Failing access in `PUBLISH_ONCE`
Throws an `Error`	The same error, unchanged	A new `InitializationError`, no cause; original error class name when available	The same error, unchanged; retry remains possible
Throws another throwable	`NoSuchElementException` with that throwable as cause	`NoSuchElementException`, no cause; original throwable class name	The original throwable; retry remains possible
Returns null	Computation failure represented by a `NullPointerException`, wrapped as above	The recorded permanent failure	Direct `NullPointerException`; retry remains possible

Waiting callers count as subsequent accesses. The failure state does not retain the original throwable. A later `InitializationError` reports an earlier failure, not a fresh occurrence of the original error condition.

The benefit over wrapping everything in `NoSuchElementException` is that ordinary `catch (Exception)` handling does not accidentally absorb subsequent observations of an `Error`. Keeping only failure information also limits retention of exception stacks, application objects, and class loaders through the failure state. The costs are a new public error type and reduced diagnostics on later access: the original cause, stack, and exact error subtype are no longer available there. `NoSuchElementException` also has a second meaning in traversal, so a lazy failure must not be interpreted as end-of-iteration.

The Javadoc cites JLS 12.4.2. The analogy is deliberately limited: class initialization propagates an initial `Error` unchanged and later reports erroneous initialization with `NoClassDefFoundError`; `Lazy` uses its own failure types and storage rules. It does not import the whole class-initialization protocol.

Same-thread recursive access to an uninitialized `COMPUTE_ONCE` slot throws `IllegalStateException`. The outer computation may catch it and succeed; it becomes sticky only if it escapes that computation. Recursive `PUBLISH_ONCE` computation need not be detected. Cross-thread dependencies can deadlock in `COMPUTE_ONCE`. No general cycle detector was added.

In `PUBLISH_ONCE`, a failing computation still throws even if another thread has installed a value in the meantime. Substituting the winner for that failure would hide an outcome of user code; the chosen rule preserves exception propagation at the cost of some accesses failing after storage has become successfully initialized.

Memory consistency: publication, without repeated volatile synchronization

The adopted cache guarantee is release publication and acquire observation, for both successful values and permanent `COMPUTE_ONCE` failures. Actions before publication happen-before actions after an access observing that publication, including actions in a handler for a recorded failure. The updater's `PUBLISH_ONCE` and `SERIALIZED` modes publish successful values with release effects and observe them with acquire effects. `PLAIN` makes no such publication guarantee, and `SERIALIZED` does not retain failures. For a segment location, `PUBLISH_ONCE` is available only when the selected layout supports the necessary aligned atomic access; `PLAIN` may also serve layouts for which atomic access is unavailable.

Alternative	Benefit	Cost / status
Volatile-access contract	Stronger ordering, including a total order of volatile accesses	Stronger than the selected publication requirement; adds obligations to repeated access (e.g., prevents @Stable access). Replaced in this branch.
Acquire/release publication	Preserves visibility of initialization while allowing later reads to be elided when publication is preserved	Does not guarantee additional ordering or visibility for subsequent mutations, or total ordering across different lazy values. Adopted.

Alternative	Benefit	Cost / status
Plain access without publication guarantees	Potentially weaker access costs	Requires a weaker user model or external synchronization. Excluded from cache factories; available as `Lazy.UpdaterMode.PLAIN` for field and array-element updates.

The distinction for discarded computations was corrected during review:

- A successful access whose own candidate loses **does acquire the winner's publication**.
- That does **not publish the losing computation's own actions to other readers** through the lazy slot.
- Actions performed by a failed `PUBLISH_ONCE` computation are likewise not guaranteed to be published to other threads through that slot.

An atomic operation is not automatically sufficient for both directions. In particular, `compareAndExchangeRelease` reads its witness with plain semantics. The updated implementations acquire-read the installed value after a losing release exchange. This is an implementation technique satisfying the contract, not an instruction sequence required of every implementation.

The revised contract permits constant folding and elision of later value or failure reads only while preserving the applicable publication guarantee. It does not promise folding on every VM, equate all final-field chains with trusted fields, or make subsequent mutation of a returned object thread-safe. Lazily obtaining an array also does not make its elements lazy. Internal volatile operations remain permissible; changing the public contract did not mean removing every volatile field from the implementation.

Caller-owned locations: the updater family

Brian's email identified an allocation-sensitive case: many cheap, short-lived objects whose caches are usually never read. Using a field already in each receiver avoids allocating a lazy holder per receiver. The updater family extends this idea to array elements and selected memory-segment values. Unlike a cache, an updater does not own the storage; the caller must supply a suitable mutable location and preserve its uninitialized-value convention.

The three short, class-token-first conveniences remain `ofStaticFieldUpdater(owner, name, computing)`, `ofFieldUpdater(owner, name, computing)`, and `ofArrayUpdater(elementType, computing)`. They use `PUBLISH_ONCE` and return `Supplier<T>`, `Function<R,T>`, and `ReferenceArrayAccessor.Computer<T>`, respectively. The advanced path uses one entry point and two terminal operations. For a direct `int[]` element:

```
import java.lang.util.Lazy;

class ArrayChoices {
    static final Lazy.IntArrayAccessor.Computer DECLARATION_SITE =
            Lazy.ofUpdater(Lazy.IntArrayAccessor.Computer.class)
                    .build((array, index) -> index + 1);
    static final Lazy.IntArrayAccessor USE_SITE =
            Lazy.ofUpdater(Lazy.IntArrayAccessor.Computer.class)
                    .buildUseSite(Lazy.IntArrayAccessor.class);
}
```

Here `Class<C>` describes the **computation shape**, including the storage coordinates and result. The returned declaration-site updater implements that same interface and retains the supplied computation. `buildUseSite(Class<U>)` returns an accessor whose first method parameter is a computation callback of the initial shape, followed by the same storage coordinates, with a compatible result. The callback is supplied per invocation and is not retained between calls. It is not an extra storage coordinate. The builder checks these shapes, the selected storage type, and the selected mode when a terminal operation builds an updater.

`UpdaterBuilder<C>` is a public interface for the API shape; its package-private implementation, `LazyUpdaterBuilderImpl` in `jdk.internal.invoke`, keeps its representation and helper state internal. Factory-returned builders remain immutable: each wither returns a new builder, leaving the original reusable. `withMode(UpdaterMode)` changes the default `PUBLISH_ONCE` mode. `withLookup(MethodHandles.Lookup)` changes the default public lookup for **field access**; private or otherwise non-public fields require suitable authority. An explicit lookup with a direct-array or segment configuration is rejected at a terminal operation because it has no field to authorize. A location selector is optional and may be chosen once; calling either `withLocation` form again is rejected:

Builder configuration	Selected storage	Required storage coordinates
No `withLocation`	One element of a directly supplied Java array	Array, followed by one or more `int` indices
`withLocation(ownerClass, fieldName)`	Mutable instance or static field, or an element reached through an array-valued field	Receiver for an instance field, then any `int` indices; only indices for a static array field
`withLocation(layout, pathElements)`	Value selected by a structured memory layout in a supplied segment	`MemorySegment`, `long` base offset, then a `long` for each open sequence coordinate

The two `withLocation` forms are alternatives, not a path from a Java field into a segment. An instance field's simple name and `ownerClass` together select its storage; a static field uses the same two arguments, with staticness obtained from the field declaration. The class token fixes loader identity. The field search may find an inherited declaration through an accessible owner, subject to the supplied lookup's access rights; ambiguous inherited declarations are rejected. This avoids the previous binary-class-name string selector and also identifies a field owner when a generic computation interface erases its receiver to `Object`. It does **not** restore generic type arguments erased from a `Class` token or prove that every `Object` produced by a raw callback is of the field's intended parameterized type. A selected reference field's value must be compatible with the accessor result, and a computed value must be storable in it. Generic arguments cannot be checked by a runtime cast; the established stricter matching rule for parameterized accessor and field types remains relevant.

Without a location selector, the updater's first storage parameter identifies the direct array type, followed by indices that traverse its dimensions in order. A declaration-site shape must have a concrete erased array parameter; a specialized use-site interface may instead fix a reference-array type inherited from its generic base. The index count may not exceed the array rank and remains subject to Java and JVM method-parameter limits. An `int...` parameter is one `int[]` parameter after erasure, not several index coordinates. No accessor expression swaps, repeats, or transforms indices. A field-backed array works the same way after the field reference is read. The selected **element**, not its array-reference field, is the lazy storage location. A final array-reference field is therefore possible even though the elements must be mutable.

When an array element is reached through an array-valued field, `PUBLISH_ONCE` and `SERIALIZED` resolve that field with acquire effects before traversing indices, whereas `PLAIN` resolves it plainly. This allows an array reference published to the field with release or volatile effects to be observed before element access; callers still must publish the array safely.

The former four advanced `ofLocationUpdater` and `ofArrayUpdater(UpdaterMode, Class, ...)` overloads separated field from array storage and declaration-site from use-site timing at the top level. The builder makes those choices composable, while providing one documented default for the common updater mode and lookup. The cost is more call steps and a second `Class` token for use-site shape validation. `build` and `buildUseSite` have distinct names to make the timing choice visible and avoid overload-resolution trouble between a callback and a shape token.

Choice	Benefit	Cost
One immutable advanced builder	One entry point and consistent defaults across fields, arrays, and segments; a configured base can be reused	Extra intermediate objects and call steps when an updater is constructed; a field's name and mode are no longer adjacent to the entry point
Explicit field-owner `Class`	Pins class-loader identity and identifies storage even when a generic computation interface erases its receiver	Adds an argument and still leaves the field name string-checked at build time
Separate `build` and `buildUseSite`	Makes declaration-site retention versus per-access callback visible; avoids ambiguous terminal overloads	Use-site form requires a second shape token and runtime shape validation
Segment path in the same builder	Reuses the timing and mode vocabulary with layout-derived coordinates	Segment aliasing and VarHandle access-mode limits rule out some otherwise available modes

The merged `Lazy` class remains a preview API: its cache factories, three updater conveniences, builder entry point, and builder operations require preview opt-in. A builder change is source- and binary-incompatible with clients of the removed advanced overloads, even if the returned updater's functional interface is unchanged.

The two field conveniences remain caller-sensitive. Each obtains lookup authority from the immediate caller and selects the supplied owner `Class`; callers cannot use them to access an unrelated private field. A wrapper calls with the wrapper's authority, whereas a builder's explicit `Lookup` is a capability the caller may delegate. The static convenience requires a mutable static field and returns a `Supplier` that retains the computation; the instance convenience requires a mutable instance field and returns a `Function` reusable across receivers. A primitive field value is boxed through those standard interfaces. The reference-array convenience selects one index from each supplied `T[]` and may serve several arrays. It accepts a reference `elementType` token, including an array class for a row-valued element, and rejects primitive or `void` element types.

`ReferenceArrayAccessor.Computer<T>` has the exact `T apply(T[] array, int index)` shape needed for that convenience. When passed to the use-site `ReferenceArrayAccessor<T>`, it computes a candidate. When returned by the declaration-site convenience, it reads, computes if needed, and returns the selected element. The outer `ReferenceArrayAccessor<T>` has the extra callback parameter on `getOrCompute`. Reusing its nested `Computer<T>` removes the briefly introduced duplicate `ObjIntFunction<T[],T>`, at the cost of giving `Computer` a second role. That removal changed the preview prototype's binary API.

Convenience	Benefit	Cost or boundary
`ofStaticFieldUpdater`	A `Supplier<T>` for one static field without an explicit lookup or custom interface	Caller-sensitive access, erased value type, one fixed mode
`ofFieldUpdater`	A reusable `Function<R,T>` whose receivers supply separate fields without one updater object per receiver	Caller-sensitive access, erased value type, one fixed mode
`ofArrayUpdater(Class<T>, ReferenceArrayAccessor.Computer<T>)`	One updater can serve many reference arrays; the element token fixes the component type	`Computer<T>` has callback and updater roles; one index and reference elements only

For example, all three conveniences can coexist in one class:

```
import java.lang.util.Lazy;
import java.util.function.Function;
import java.util.function.Supplier;

final class Example {
    private static String shared;
    private String derived;

    static final Supplier<String> SHARED =
            Lazy.ofStaticFieldUpdater(Example.class, "shared", () -> "ready");
    static final Function<Example, String> DERIVED =
            Lazy.ofFieldUpdater(Example.class, "derived", e -> "value");
    static final Lazy.ReferenceArrayAccessor.Computer<String> ELEMENT =
            Lazy.ofArrayUpdater(String.class, (array, index) -> "item " + index);
}
```

The builder expresses the equivalent advanced field shape with an explicit computation interface. A boxed `Double` permits a computed positive zero because `null`, rather than `+0.0`, is the reserved uninitialized value:

```
class Point {
    private final double x, y;
    private Double radius;

    Point(double x, double y) { this.x = x; this.y = y; }
    interface Radius { Double getOrCompute(Point point); }

    private static final Radius LAZY_RADIUS = Lazy.ofUpdater(Radius.class)
            .withLookup(MethodHandles.lookup())
            .withLocation(Point.class, "radius")
            .build(point -> Math.hypot(point.x, point.y));

    double radius() { return LAZY_RADIUS.getOrCompute(this); }
}
```

The lookup authorizes the field access; the `Point.class` token selects it without asking a loader to resolve a binary-name string. Direct-array access needs no caller-supplied lookup and still supports a non-public computation interface. A raw generic computation token can be admitted when `ownerClass` supplies the field owner, but `Function<Point,Double>.class` cannot exist: Java erases the type arguments in a `Class` token. A non-generic, type-fixing interface remains the clear way to obtain precise static typing of a custom computation shape and value.

The use-site form exposes a callback per access. A built-in field accessor can fix the receiver type in a concrete subinterface:

```
class Counter {
    private int seed = 41;
    private int cached;

    interface CounterAccessor extends Lazy.IntFieldAccessor<Counter> { }

    private static final CounterAccessor LAZY = Lazy.ofUpdater(ToIntFunction.class)
            .withMode(Lazy.UpdaterMode.SERIALIZED)
            .withLookup(MethodHandles.lookup())
            .withLocation(Counter.class, "cached")
            .buildUseSite(CounterAccessor.class);

    int value() { return LAZY.getOrCompute(counter -> counter.seed + 1, this); }
}
```

The `ToIntFunction` token loses its generic receiver, but the explicit `Counter.class` identifies the storage owner and the concrete `CounterAccessor` fixes the use-site API. The callback runs only while `cached` is zero; the first successful result is retained. A later callback can retry after a failure in `SERIALIZED` mode. The builder and returned use-site accessor do not retain an invocation's callback after it completes.

A static field or static array field uses the same builder, differing only in coordinates:

```
package example;

class Answer {
    private static int value;
    private static final IntSupplier LAZY_VALUE = Lazy.ofUpdater(IntSupplier.class)
            .withLookup(MethodHandles.lookup())
            .withLocation(Answer.class, "value")
            .build(() -> 42);
}

class Holder {
    private static final int[] ints = new int[8];
    private static final int[][] matrix = { new int[4], new int[4] };

    private static final IntUnaryOperator LAZY_INTS = Lazy.ofUpdater(IntUnaryOperator.class)
            .withLookup(MethodHandles.lookup())
            .withLocation(Holder.class, "ints")
            .build(index -> index + 1);

    private static final IntBinaryOperator LAZY_CELLS = Lazy.ofUpdater(IntBinaryOperator.class)
            .withMode(Lazy.UpdaterMode.SERIALIZED)
            .withLookup(MethodHandles.lookup())
            .withLocation(Holder.class, "matrix")
            .build((row, column) -> 1 + row * 4 + column);
}
```

`LAZY_INTS.applyAsInt(3)` selects `ints[3]`, and `LAZY_CELLS.applyAsInt(1, 2)` selects `matrix[1][2]`. A null intermediate array or invalid index fails before computing. Each final physical array element is its own location.

Direct arrays omit `withLocation`. A declaration-site updater implements the computation shape itself, while a use-site updater takes the same computation interface first:

```
int[] entries = new int[8];
Lazy.IntArrayAccessor.Computer eagerChoice =
        Lazy.ofUpdater(Lazy.IntArrayAccessor.Computer.class)
                .build((array, index) -> index + 1);
Lazy.IntArrayAccessor perCallChoice =
        Lazy.ofUpdater(Lazy.IntArrayAccessor.Computer.class)
                .buildUseSite(Lazy.IntArrayAccessor.class);

int first = eagerChoice.apply(entries, 0);
int second = perCallChoice.getOrCompute(
        (array, index) -> array.length + index + 1, entries, 1);
```

A stored nonzero value is returned without invoking the callback. In `PUBLISH_ONCE`, candidate computations can race and successful accesses return the installed winner; a losing candidate is discarded. The nine ready-made use-site interfaces cover static (`StaticIntFieldAccessor`, `StaticLongFieldAccessor`, `StaticReferenceFieldAccessor<V>`), instance (`IntFieldAccessor<R>`, `LongFieldAccessor<R>`, `ReferenceFieldAccessor<R,V>`), and one-dimensional direct-array

(`IntArrayAccessor`, `LongArrayAccessor`, `ReferenceArrayAccessor<E>`) forms. Their outer methods are named `getOrCompute`; nested array `Computer` methods retain `apply`. An explicit field owner permits raw generic field-accessor tokens, though a non-generic specialization is recommended for source type safety and parameterized value-type checking. A direct reference-array use-site accessor does need a non-generic specialization, such as `interface Names extends ReferenceArrayAccessor<String> { }`, to identify its array type. Custom functional interfaces cover other primitive types and higher array ranks.

For `int[][][]`, one index selects an `int[][]` plane, two select an `int[]` row, and three select an `int` cell. A direct-array updater can choose each rank-specific computation shape:

```
class Volume {
    interface Plane { int[][] getOrCompute(int[][][] array, int plane); }
    interface Row { int[] getOrCompute(int[][][] array, int plane, int row); }
    interface Cell { int getOrCompute(int[][][] array, int plane, int row, int column); }

    private static final Plane PLANES = Lazy.ofUpdater(Plane.class)
            .build((array, plane) -> new int[3][]);
    private static final Row ROWS = Lazy.ofUpdater(Row.class)
            .build((array, plane, row) -> new int[4]);
    private static final Cell CELLS = Lazy.ofUpdater(Cell.class)
            .build((array, plane, row, column) ->
                    1 + plane * 12 + row * 4 + column);

    private final int[][][] volume = new int[2][][];

    int cell(int plane, int row, int column) {
        PLANES.getOrCompute(volume, plane);
        ROWS.getOrCompute(volume, plane, row);
        return CELLS.getOrCompute(volume, plane, row, column);
    }
}
```

The outer calls install non-null planes and rows before the cell updater traverses them. In `SERIALIZED` mode, coordination follows the final container array and index, regardless of which outer array or owner field reached it. Compatible updaters sharing a location must use the same mode and interchangeable computations. Neither the builder nor the updater eagerly adds a holder to each receiver or array element; `SERIALIZED` can allocate auxiliary weakly keyed coordination state when needed.

A segment location uses a `MemoryLayout` path ending at a `ValueLayout`. Its updater coordinates match the layout-derived `VarHandle`: `MemorySegment`, `long` base offset, and one `long` for each open sequence element. For example (imports omitted):

```
class SegmentExample {
    interface SegmentAnswer {
        int getOrCompute(MemorySegment segment, long baseOffset);
    }

    static final MemoryLayout LAYOUT = MemoryLayout.structLayout(
            ValueLayout.JAVA_INT.withName("answer"));
    static final SegmentAnswer ANSWER = Lazy.ofUpdater(SegmentAnswer.class)
            .withLocation(LAYOUT,
                    MemoryLayout.PathElement.groupElement("answer"))
            .build((segment, baseOffset) -> 42);
}
```

The segment and offset are supplied when `SegmentExample.ANSWER.getOrCompute(segment, 0L)` is called. A non-default value already in the selected memory is returned without computing. The layout path must end at a value layout; dereference path elements are rejected. `PUBLISH_ONCE` requires an aligned selected layout and the acquire/atomic-release access modes needed by the selected carrier. The builder rejects that mode at `build` or `buildUseSite` when the layout-derived VarHandle cannot support it; it does not silently fall back to `PLAIN`. An unaligned value layout may work with `withMode(PLAIN)` if plain read and write are supported. `SERIALIZED` is rejected for segments because two segment objects may alias the same bytes, defeating a lock keyed only by segment-object identity. Segment access continues to enforce scope lifetime, thread ownership, bounds, alignment, and read-only checks on every invocation. For address layouts, the zero address (`MemorySegment.NULL`), rather than Java `null`, is the reserved uninitialized value; a `null` candidate is also rejected because it is not a valid stored address.

An address layout stores an address, not the candidate `MemorySegment` object. After successful `PUBLISH_ONCE` publication, the updater rereads the location. All successful `PUBLISH_ONCE` accesses return layout-materialized segments with the installed native address that compare equal by `MemorySegment.equals`; they need not return the candidate's object identity, scope, or size. A computing access in `PLAIN` mode instead returns its own candidate after writing it. The caller must keep memory reachable through a stored address alive for as long as that address may be used; the updater's stored address does not retain the pointee segment or manage its lifetime.

All updater entry points use the selected location's default value as the uninitialized sentinel: `null`, `false`, integral zero, positive floating-point zero, or the zero address for an address layout. A computation returning that sentinel throws `IllegalArgumentException` and leaves the location uninitialized; a later access may retry. A preexisting non-default value is returned without computing. A selected field must not be `final`, though a final array-reference field can lead to mutable elements. A null-restricted reference field is rejected when the updater is built, and a null-restricted reference array element cannot provide the required `null` sentinel. Callers must safely publish receivers, arrays, segments, and preinitialized values and must not reset selected locations or change intermediate array references while an updater is in use. `PLAIN` allows racy overwrites and no safe publication; `PUBLISH_ONCE` offers one installed winner with acquire/release publication; `SERIALIZED` allows one concurrent computation per Java field or array element, with retry after failure.

Path not taken: cracking accessor lambdas

An earlier revision supplied a second lambda to select storage, for example `point -> point.radius`, `index -> Holder.ints[index]`, or `(array, row, column) -> array[row][column]`. This made the selector type-checked by the Java compiler, supported IDE rename tools, and avoided string field names. It also let one factory, then called `ofLocation`, cover fields and direct arrays. The prototype inspected generated lambda implementation bytecode without executing the accessor.

That advantage came with a complex and fragile contract. The implementation had to recover the lambda's implementation method, inspect bytecode and constant-pool references, recognize array-load opcodes and method-handle forwarding shapes, and then recheck lookup authority. Captures, method references, transformed indices, and several compiler or hidden-class shapes needed exclusions or special cases; custom class loaders complicated access to implementation bytes. Even simple source expressions depended on the exact bytecode shape emitted by the compiler. The mechanism also allowed reversed index arguments, adding a semantic choice whose only expression was the selector lambda. Those costs outweighed the convenience of a type-checked field expression for this prototype.

Storage-selection choice	Pros	Cons / decision
Named field with `String`, owner `Class`, and `Lookup`	Simple selector grammar; no inspection of compiler-generated accessor code; explicit class-loader identity and access authority	Field names are checked when the updater is built rather than by the Java compiler. Adopted through `ofUpdater(...).withLocation(ownerClass, fieldName)`; the two field conveniences derive a caller lookup.
Direct array identified by a computation or use-site shape	No selector string or storage-selection callback; coordinates have one order	A declaration-site builder needs a concrete erased array parameter. A specialized use-site accessor can instead identify a reference array through its fixed type argument. Neither uses `withLocation`; the one-index reference-array convenience takes an element-type token.
Cracked accessor lambda	Compiler type-checks the storage expression and IDE renames can update it	Depends on generated bytecode and rejects several otherwise ordinary lambda shapes. Dropped.

The current design uses a string only where a field genuinely needs a name and omits a selector for a direct array because the computation shape already identifies it. The explicit owner `Class` avoids binary-name resolution through the lookup's loader. This removes accessor bytecode inspection from the public contract and implementation path. The computation remains a normal lambda or method reference; it may capture state. A declaration-site updater retains it, while a use-site updater receives it on each access. The design gives up compile-time checking and automatic refactoring of field names; a wrong name is detected when an updater is built. A receiver type alone cannot identify which of its fields is the desired storage location.

Collections: retain ordinary shapes, specify the lazy boundary

The collection factories create unmodifiable collections. Every factory invocation supplies independent initially uninitialized storage; this does not require distinct result objects if callbacks return the same object. Views and map entries share their originating storage. Empty-instance reuse is permitted when construction has size zero, no keys, or no candidates; this permission does not turn a nonempty-candidate set that later filters to empty into shared lazy state.

Shape	What is fixed or copied eagerly	What is computed lazily	Main benefit / cost
List	Size and index domain	One element per index	Useful indexed interface and `RandomAccess` views; element-inspecting operations can trigger work or fail
Map	Copied key set	One value per key	Keys and size can be used without evaluating values; ordinary value-oriented operations can evaluate them
Set	Copied candidate set	One membership decision per candidate, retaining both true and false	Supports lazy filtering with stable membership; size and hash require all membership statuses

`ofSetCache` was added in this branch beyond Brian's final sketch. It computes membership of existing candidates, not new element objects or an unbounded stream.

List sublists and reversed views share original slots and compute with the original indices. Looking up an absent map key or an object outside a set's candidate domain does not invoke its computing function. An out-of-range list access fails before computation.

Map computations receive the **retained key from construction**, even when queried with a different but equal object. Set predicates receive the retained candidate on the same basis. The copies are shallow: the key/candidate objects themselves are retained, not cloned. This removes dependence on which equal query object arrives first. Its cost is that information present only on a query object is intentionally not an input to the cached computation. Input sets must use equality-compatible membership; changing key/candidate equality or hash codes while retained is outside the contract. Map and set iteration order is unspecified.

Copying a lazy input set can evaluate **the input's memberships** and propagate its failures. Construction does not thereby compute the output map's values or the output set's memberships. This distinction preserves snapshot semantics while making clear that accepting a lazy input is not a promise of a computation-free factory call.

The following stronger collection choices were made:

Decision	Pros	Cons
Null results are computation failures; null keys/candidates are rejected during construction	Clear distinction between uninitialized storage and a retained result	Absent computed values need an explicit representation such as `Optional`
Unsupported mutations always throw, even for no-ops, without invoking computing functions or supplied mutation callbacks	Predictable unmodifiability; `sort`, `replaceAll`, `removeIf`, and map updates cannot accidentally perform work	More overrides than permissive default methods; no “successful” empty mutation
Entries remain valid after traversal and share the original per-key storage	Entries can safely escape an iterator; later access observes the same result or permanent failure	Retaining one entry can retain the map and other unresolved state
Returned suppliers, functions, and collections are not serializable, including empty collections	Avoids defining serialization of callbacks, incomplete state, and sticky failures	Applications must choose their own materialized or domain-specific representation
Scalar `equals`, `hashCode`, and `toString` do not compute; collection content operations may compute	Keeps scalar diagnostics harmless while retaining ordinary collection meanings	Collection logging, hashing, equality, or copying can run user code and fail

Obtaining map entries or their keys does not itself compute mapped values. Reading an entry's value does; `Map.Entry.copyOf` therefore may compute or report a failure while producing its detached copy. Unmodifiable collections can still contain mutable values. In particular, a lazy map's structural hash must reflect current value hashes rather than be permanently cached merely because its storage locations never change.

Traversal and streams: failures do not silently remove data

When the traversal itself encounters a lazy computation failure, the pending value or candidate is not consumed. A failed list-iterator `next` or `previous` leaves its cursor unchanged. Set iterators may advance past candidates already known to be absent, but keep a failing membership pending. The rules cover bulk traversal, `hasNext`/`hasPrevious`, and spliterators; a failed source access in `tryAdvance` does not invoke its action. A split that fails because of lazy computation must preserve the original's remaining coverage.

The benefit is that retry in `PUBLISH_ONCE` cannot silently skip data, while `COMPUTE_ONCE` repeatedly reports the same slot's recorded failure. The cost is extra traversal state and implementations beyond ordinary defaults. This is not transactional rollback: earlier values, memberships, and failures remain retained, and elements already delivered remain consumed.

Failures thrown by a traversal action follow the ordinary interface's continuation rules, including when that action makes its own failing lazy access. Such a failure must not be mistaken for failure of the traversal's own source access. More generally, user callbacks outside the computing function do not themselves create an additional initialization failure; lazy accesses inside them retain their own rules.

Creating streams or spliterators and querying their metadata do not compute values or report stored computation failures. Source spliterators, including those produced by splitting, report `NONNULL` and `IMMUTABLE`; lists and views additionally report `ORDERED`, `SIZED`, and `SUBSIZED`. Map key/entry views report `DISTINCT`, `SIZED`, and `SUBSIZED`; value views report the latter two. Lazy sets report `DISTINCT` and need not report an exact size before it is known without evaluation.

The design does not promise minimal evaluation by every traversal, split, or parallel operation. Short-circuiting can compute more than the final result consumes. Conversely, a sized `count()` can avoid element computation altogether, even when some element has a recorded failure. This preserves stream optimization freedom. A terminal operation such as `count()` is not evidence that every lazy value has been evaluated; validation must explicitly traverse or access the relevant values. The current implementation's non-computing range splits are stronger than the minimum public guarantee and are not a new API commitment.

Execution, retention, and lifetime

Computations run synchronously on the accessing thread. `COMPUTE_ONCE` coordinates one slot; a shared collection callback may still execute concurrently for different slots. Waiting is uninterruptible and preserves an interrupt already present or received during the wait, on normal or abrupt completion. A computing function's own interruption handling is not constrained by that waiting rule. Cancellation, timeouts, and asynchronous execution are not provided.

This keeps access and exception propagation direct and avoids an executor policy. It also means that a blocking computation blocks its caller, and potentially other callers waiting for its result. Neither mode provides a general liveness guarantee.

Cache factories strongly retain the computing function while any slot remains unresolved, and release the invocation reference when all slots have either a result or permanent failure. Mixed successful and failed slots count as terminal. A view's lifetime is assessed against the whole original storage, including slots outside that view. Declaration-site `Lazy.ofStaticFieldUpdater`, `ofFieldUpdater`, and `ofArrayUpdater` conveniences, and builder-created `build(computingFunction)` updaters, retain their computing functions for their lifetimes, even when a static-field updater selects one field that has been initialized. Instance-field, array, and segment updaters can encounter new locations later. Use-site updaters do not retain a computation between accesses; a callback supplied to an access can still be held while that invocation is in progress.

This rule bounds retention after completed initialization while ensuring that unresolved slots remain computable. It cannot guarantee garbage collection: values, keys, candidates, in-flight computations, and other application references may still retain the callback. `Lazy.ofScalarCache(CacheMode, Function)` does not otherwise retain its context, and the updater factories do not otherwise retain receivers, arrays, or segments; callbacks or results can themselves contain references. The updater's `SERIALIZED` mode uses weak receiver or selected-container-array keys for its auxiliary coordination state.

Records and Valhalla: supported patterns and limits

The discussion tested the derived-property use case for `record Point(int x, int y)`. That exact record cannot declare extra instance fields for `radius` and `phi`; see JLS 8.10.2. A static `ofScalarCache(CacheMode, Function)` would incorrectly share one slot across points. An ordinary class can declare per-instance holder or updater storage instead. Adding holders as record components would change its representation and generated equality; no transparent record cache facility or new language feature was implemented.

For the Valhalla value-class model examined here, a final field can reference a mutable lazy holder, and `ofScalarCache(CacheMode, Function)` with a noncapturing callback avoids capturing incompletely initialized `this`. A value-class instance field cannot itself serve as mutable storage for `Lazy.ofUpdater(...).withLocation(valueClass, fieldName)` because value-class instance fields are final. A final field referencing a mutable array could in principle supply lazy array elements through such a field-backed builder or pass that array directly to a no-location builder, subject to the value-class and array rules; this combination has not been validated here. The important cost is equality: holder or array references participate in value-object == and inherited `Object.equals`, so objects with otherwise equal application data may compare unequal. Custom `equals`/`hashCode` can define logical equality but cannot change ==. These points concern the Valhalla model, whose draft specification remains a separate language-development effort.

The benefit is a workable library pattern without adding language machinery. The costs are holder storage and possible equality consequences. The API does not make all capturing lambdas illegal, guarantee allocation-free value-class caching, or supply an indexed-relative collection shape whose ordinary `List.get` accepts an owning value object.

Alternatives outside the adopted scope

The distinctions in this table matter: absence from the current surface is not proof of permanent rejection.

Alternative	Status in this work	Reason or unresolved tradeoff
Lazy fields / cached methods as new language features	Discussed in Brian's background; not implemented here	Static and instance use cases do not divide cleanly; records and value classes add separate constraints
Public stability annotation or general stable VarHandle/MethodHandle toolkit	Not exposed by this API	The high-level entry point targets common usage; possible lower-level facilities remain separate work
Full coordinate family beyond array indices	Present in earlier sketches; the updater layer supports any feasible array rank, optionally through one owner field or a static array field, but no general indexed-relative coordinate type	A smaller API loses non-array context-plus-index use cases and arbitrary coordinate transformations
Context class-token overload	Explored, not adopted	Better inference cues versus extra signature surface
Further primitive specializations	Pre-canned use-site `int` and `long` accessors cover one-dimensional arrays, instance fields, and static fields; reference counterparts now cover those same shapes, but other primitive types are not provided here	Custom updater and callback interfaces remain available, while a larger built-in family would increase the API surface
Null as a successful lazy result	Not offered	Non-null retained values simplify the contract; absence needs another representation
Reset, invalidation, eviction, or per-argument memoization	Not provided	These require a lifecycle beyond one retained result per storage location
Explicit initialization-state query or non-computing peek	Not provided	Standard return interfaces expose access, not a new lifecycle protocol
Serialized retry in cache factories	Not provided; `Lazy.UpdaterMode.SERIALIZED` offers it for field and array-element storage	Requires waiting and auxiliary coordination state
Independent policy flags	Not provided	The design deliberately stops short of the full policy/storage product
`COMPUTE_ONCE` for caller-owned updaters	Not provided by `Lazy.UpdaterMode`	At-most-once computation with permanent failure would require sticky failure state in addition to the selected field or array element
Dedicated segment-backed `Lazy.ofSegment` overloads	Experimental overloads removed from the earlier prototype; the builder now supports selected value layouts through `withLocation(layout, pathElements)`	A single builder avoids another factory family, but cannot support `SERIALIZED`, dereference paths, or unsupported atomic access modes without additional contracts
Cancellation, timeouts, executor-backed evaluation	Not provided	No scheduling or task-management policy is added to lazy access
Removal or deprecation of `LazyConstant` and collection `ofLazy` factories	Not performed	Migration and compatibility policy for those earlier APIs require a separate platform decision

Implementation and validation status

The implementation work followed the specification reviews. It updated scalar, field, and collection publication; retained stable storage opportunities; added failure-preserving traversal and views; preserved original storage through map entries and key views; corrected equality exception propagation; and preserved publication through cached set size/hash results. Existing legacy factory behavior was kept on its existing paths.

The source layout has one public facade, `java.lang.util.Lazy`, exported by `java.base`. It validates arguments and delegates to internal implementations. Scalar and collection behavior lives in `jdk.internal.util` (`LazySupport` and `LazyCollections`); the package-private `LazyUpdaterBuilderImpl` and the other updater support classes (`LazyUpdaterSupport`, `LazySegmentUpdaterSupport`, and `LazyUpdaterFactory`) live in `jdk.internal.invoke` and handle updater configuration, field selection, coordination, segment access, and functional-interface adapters. The existing `List.ofLazy`, `Map.ofLazy`, and `Set.ofLazy` factories also use the internal collection implementation. Generated hidden classes may still implement updater interfaces; that adapter mechanism is separate from the dropped accessor-bytecode inspection. The builder's `withLookup` governs access to a selected Java field, with a public lookup by default; `ofStaticFieldUpdater` and `ofFieldUpdater` derive a caller lookup through the internal access bridge. Direct-array updaters need no field resolution. One- and two-index updaters retain specialized call paths. Higher-rank updaters collect their indices into an `int[]` before traversal, with a potential per-access allocation cost.

The validation results below distinguish the current builder revision from earlier prototypes. Historical counts use their then-current class names and test paths; they do not validate the builder or segment extension.

The builder revision passed the `java.base-java` and `docs-jdk-api` builds. The Lazy runtime suites passed 235 updater and 327 cache JUnit executions. After the golden negative-compilation diagnostic changed from `class` to `interface`, its focused rerun passed (1/1). `javap` confirms that `Lazy.UpdaterBuilder` is a public interface and `jdk.internal.invoke.LazyUpdaterBuilderImpl` is package-private. These checks establish the compiled API shape and covered behavior, not performance or every possible memory-model execution.

The earlier validation sequence built `java.base` and passed 13 targeted jtreg test descriptions, comprising 821 successful JUnit invocations plus non-JUnit checks. Coverage included the new Lazy suites, legacy lazy-list/map/set tests, a negative compilation test, lookup/module tests, and MOAT collection checks extended to both coordinated modes. One new test file initially needed a generic assertion typing correction and then passed on a focused rerun; the combined results were all passing. Those recorded results predate the mode-interface and three-mode updater extension and do not validate that extension.

The preceding mode-interface variant was validated against the built JDK: the `java.base` and JDK API Javadoc builds passed; all 12 Lazy jtreg descriptions passed (376 JUnit executions), including compile-fail checks for the updater-only mode; and both `LazyUpdaters` jtreg descriptions passed (82 JUnit executions across normal and interpreter runs). The earlier MOAT run also passed; it was not repeated for that variant. Those results do not validate the independent enums or `SERIALIZED` retry behavior introduced by this revision.

The internal-package migration passed a clean JDK `images` build. On that build, 17 focused jtreg descriptions passed (904 JUnit executions), covering the independent modes, updater retry and field access, lazy factories and collections, and legacy list/map/set behavior. The throwable class-unloading and MOAT collection checks also passed as two additional jtreg descriptions.

After removal of the `Lazy.ofField` and `Lazy.ofSegment` overloads and their tests, and the updater-mode renaming, the pre-array revision passed the `java.base-java` and JDK API Javadoc builds. All 11 targeted jtreg descriptions passed: eight under `java/util/Lazy` and three under `java/lang/invoke/LazyUpdaters`, including 408 JUnit invocations. These runs establish the tested behavior, not a proof of all memory-model executions, measured allocation savings, or a performance comparison across VMs. The outstanding platform decisions remain adoption and release sequencing, the relationship to the earlier APIs, and whether workload evidence warrants any additional shapes or specializations.

For the earlier lambda-selector array-element extension, the `java.base-java` and JDK API Javadoc builds passed. All three `LazyUpdaters` jtreg descriptions passed, including 104 JUnit executions across normal and interpreter runs. Those tests covered one- and two-index direct and owner-field accessors, swapped indices, every primitive array-load opcode and reference elements, the three updater modes, aliasing of inner arrays, reserved-value retries, bounds and null failures, and lookup checks. The `java/util/Lazy` tests were not rerun for that extension, which did not change the `Lazy` implementation.

In that earlier lambda-selector design, index-only static-array access was folded into `ofLocation` and an earlier, different `ofArray` factory was removed. The updated JDK image and JDK API Javadoc build passed. The three focused `LazyUpdaters` jtreg descriptions (`OfStaticArrayLocationTest`, `OfLocationTest`, and `FindVarHandleTest`) passed with 130 JUnit invocations across normal and interpreter runs. They covered static one- and two-index accessors, reversed

indices, the default and explicit modes, lookup checks, and the existing receiver-based forms. The later direct-array factory, now named `ofArrayUpdater`, has different parameters and semantics.

For the `LazyCaches` and `ofScalar` rename, the JDK image and JDK API Javadoc builds passed. All eight jtreg descriptions under `test/jdk/java/util/Lazy` passed (326 JUnit invocations), as did the MOAT collection test and the `LazyUpdaters` mode-mismatch compile-fail test. The test directory retained its historical `Lazy` name; at that stage, its source exercised `LazyCaches`. These results also precede the current selector redesign.

For the initial string-selector and direct-array design, before `ofArray` dropped its lookup parameter, the `java.base-java`, full JDK `images`, and JDK API Javadoc builds passed. All four focused `LazyUpdaters` jtreg descriptions and the updated `LazyDemo` passed, including 101 JUnit invocations across normal and interpreter runs and the mode-mismatch compile-fail check. The tests cover named instance and static fields, direct and field-backed arrays, row-valued selections, natural index order, the three modes, lookup access, inherited fields reached through public subclasses and interfaces, ambiguous-name rejection, and distinct static fields with the same binary name in separate class loaders. An initial static-array test expected an access error from `publicLookup`, but that lookup could not load the application class; the test was corrected to use a restricted lookup in the same loader and passed on rerun. These results validate the focused behavior, not performance claims or all possible memory-model executions.

After removing the lookup parameter from `ofArray`, the `java.base-java` and JDK API Javadoc builds passed. All four focused `LazyUpdaters` jtreg descriptions passed, including 101 JUnit executions across normal and interpreter runs; a test-only naming cleanup was followed by another successful `OfLocationTest` run. At that stage, the suite covered a direct array with a private functional interface and a private element type, while retaining `ofLocation` lookup-access tests. These checks do not establish performance or all possible memory-model executions.

For the arbitrary-rank extension, the `java.base-java` and JDK API Javadoc builds passed. All four focused `LazyUpdaters` jtreg descriptions passed, including 115 JUnit executions across normal and interpreter runs. New tests cover three- and four-index direct access, three-index instance and static array fields, higher-rank partial indexing, aliases, failures before computation, retries after a reserved result, and rejection when the index count exceeds array rank. These are representative ranks, not an exhaustive test of every JVM-legal method arity or memory-model execution.

Before adding the reference pre-canned interfaces, the declaration-site and use-site design passed the JDK API Javadoc build and all five `LazyUpdaters` jtreg descriptions, including 153 JUnit executions across normal and interpreter runs. The use-site tests covered direct and field-backed arrays, static and instance fields, the six pre-canned `int` and `long` shapes, all three modes, shared physical-element coordination, generic receiver specialization, callback validation, reference subtype results, primitive widening, and diagnostics for erased generic interface tokens and storage coordinates. This validates the exercised behavior at that stage, not the subsequently added reference interfaces, performance, or every memory-model execution.

With the three reference pre-canned interfaces, the `java.base-java` and JDK API Javadoc builds passed. All five `LazyUpdaters` jtreg descriptions passed, including 167 JUnit executions across normal and interpreter runs. The added tests cover all three updater modes for direct reference arrays and static and instance reference fields, cached and reserved-null results, array covariance, raw generic-token diagnostics, incompatible field types, and matching or mismatching parameterized field types through direct and indirect specializations, including wildcard bounds. These tests do not establish performance or every memory-model execution.

After requiring an explicit updater `Mode` and moving `@PreviewFeature` to the `LazyUpdaters` class, the `java.base-java` and JDK API Javadoc builds passed. All five `LazyUpdaters` jtreg descriptions and `LazyDemo` passed, comprising 169 JUnit executions across normal and interpreter runs. A client compilation using the built JDK rejected `LazyUpdaters` without preview enabled and succeeded with `--enable-preview`. These checks validate the migrated call sites and preview boundary, not source or binary compatibility with the removed overloads.

For the final two-facade documentation reconciliation, the JDK API Javadoc build passed, followed by all five `LazyUpdaters` and eight `LazyCaches` jtreg descriptions (494 JUnit executions). All 13 examples in the two class Javadocs were compiled individually and exercised with the built preview JDK. A separate value-class probe confirmed the documented rejection timing for null-restricted reference fields and array elements in all three updater modes. These checks establish that those examples were usable and the focused regressions passed; they do not validate the later one-class rename or prove every memory-model execution or measure performance.

For this one-class move, the `java.base-java`, full JDK `images`, and JDK API Javadoc builds passed. Strict direct Javadoc validation and compilation of all 13 snippets in the merged `Lazy` class also passed. The refreshed image exports `java.lang.util` and contains `Lazy`, without either former public facade. All 24 targeted jtreg descriptions passed: eight cache tests (327 JUnit executions), five updater tests (167), the MOAT collection test, and ten legacy `LazyConstant` tests (477). The two negative-compilation tests matched diagnostics for the intended `CacheMode`/`UpdaterMode` type errors. A client

using both factory families compiled and ran with preview enabled; compilation without preview correctly rejected the API. These checks validate the moved surface and representative behavior, not performance or every memory-model execution.

For the static-field convenience (then named `ofFieldUpdater`, now `ofStaticFieldUpdater`), the `java.base-java`, JDK `images`, and JDK API Javadoc builds passed. The focused `OfLocationTest` passed with 102 JUnit executions across normal and interpreter runs, covering public and same-nest private fields, unrelated-private denial, a malformed field name, reflective and method-handle invocation, and two custom loaders holding classes with the same binary name. Separate one-off probes confirmed that a caller in a named module with a package not opened to `java.base` can initialize its own private static field, that an external caller cannot invoke the internal lookup bridge to obtain a forged full-privilege lookup, and that method-handle invocation can lose cross-package protected subclass access even when a direct call succeeds. These checks validate the exercised access paths, not every class-loader or caller-sensitive invocation arrangement.

For the earlier static rename, instance-field convenience, initial `ObjIntFunction` form, and reference-array convenience, the `java.base-java` and JDK API Javadoc builds passed. The focused `OfLocationTest` passed with 118 JUnit executions across normal and interpreter runs. New cases cover per-receiver and per-element storage, private caller fields, primitive field boxing, preexisting values, reserved-value retries, private reference components, array-valued elements, bounds, and rejection of primitive or `void` element-type tokens. Existing static tests also exercised the renamed entry point through direct, reflective, and method-handle invocation. These checks predate the reuse of `ReferenceArrayAccessor.Computer<T>` and do not establish performance or every possible class-loader, caller-sensitive, or memory-model execution.